
Simulador K-Means

Municipios de Colombia (IDF 2022)

Documentación del proyecto: arquitectura, dependencias,
despliegue y proceso de desarrollo

Autor: David Bello

Septiembre de 2026

Repositorio: github.com/daesbello/kmeans-municipios-colombia

Aplicación en producción: kmeans-municipios-colombia.streamlit.app

1. Resumen — qué se hizo

Se construyó un **simulador interactivo de k-means** sobre los municipios de Colombia, replicando la metodología del estudio de MinTIC (2023) *"Plazos para la implementación del Marco de Referencia de Arquitectura Empresarial"*, que originalmente usó k-medias para agrupar 1.101 municipios según variables socioeconómicas. La aplicación se construyó de forma iterativa, conversacional, con asistencia de IA (Claude Code), partiendo del dataset y el PDF de metodología hasta llegar a una app web pública, documentada y con su propio historial de decisiones.

La aplicación final incluye:

- Un módulo introductorio ("Cómo funciona") que explica k-means con un ejemplo mínimo de 24 puntos en 2D, mostrando lado a lado el código real de cada paso y su efecto gráfico.
- Un simulador interactivo que ejecuta k-means paso a paso sobre los 1.101 municipios (inicialización, asignación, actualización, convergencia), con controles de reproducir/pausar/avanzar o una barra para saltar a cualquier paso.
- Para el paso actual, todo se recalcula en vivo e integrado: gráfica de clusters (PCA 2D), métricas de calidad (inercia, silhouette, Davies-Bouldin, Índice de Dunn), perfil y tamaño de cada cluster, y matriz de confusión contra dos esquemas de referencia (categorías oficiales del IDF y una réplica del estudio MinTIC), con accuracy, ARI, NMI y precisión/recall/F1.
- Un análisis agregado (curva del codo e Índice de Dunn) para justificar la elección de $k=3$.
- Documentación integrada dentro de la propia app, más tres documentos en el repositorio: README, INFORME (metodología y resultados) y ARQUITECTURA (este contenido, en detalle).

2. Enlaces

Repositorio de código (GitHub)	https://github.com/daesbello/kmeans-municipios-colombia
Aplicación en producción	https://kmeans-municipios-colombia.streamlit.app/
Plataforma de despliegue	Streamlit Community Cloud (gratuita) — redespiegue automático con cada <code>git push</code> a <code>main</code>

3. Capturas de la aplicación

3.1 Módulo "Cómo funciona" (ejemplo sencillo)

Antes del simulador con datos reales, un ejemplo mínimo (24 puntos en 2D, $k=3$) muestra el código Python real de cada paso junto a su efecto gráfico — y usa el mismo k / `random_state` elegidos en la barra lateral, así que cambiar la configuración también cambia este ejemplo.

Controles del simulador

Variables de caracterización

- Número de habitantes
- Índice de Desempeño...
- Ingresos Totales Muni...
- Puntaje IDF (Índice d...
- Densidad poblacional

Municipios con datos faltantes (21 Áreas No Municipalizadas sin IDF)

- ☒ Excluir del análisis
- ☐ Imputar con la mediana

☒ Estandarizar variables (StandardScaler)

Número de clusters (k)

3

El estudio MINTIC 2023 usó $k=3$, elegido maximizando el Índice de Dunn.

> Parámetros avanzados

Referencia para matriz de confusión

Categorías oficiales IDF (DNP)

Desarrollado por David Bello

Simulador K-Means — Municipios de Colombia

Dataset: Consolidado de municipios con IDF 2022 (DNP) · 1101 de 1123 municipios usados (22 con dato faltante, estrategia: excluir del análisis).

[Cómo funciona \(ejemplo sencillo\)](#) [Simulador interactivo](#) [Documentación](#)

K-means en 4 pasos, con un ejemplo mínimo

Antes de meterse con los 1.101 municipios (5 variables), veamos exactamente qué hace k-means con 24 puntos en 2 dimensiones — el mismo código, a menor escala, para que se vea claro.

Usa el mismo $k=3$ y `random_state=42` que elegiste en 'Controles del simulador' — cámbialos ahí y este ejemplo se recalcula también. (Los 24 puntos en sí se mantienen fijos, solo cambia cómo k-means los agrupa.)

1. Los datos: 24 puntos en 2D, agrupados en 3 nubes

```
from sklearn.datasets import make_blobs

X, _ = make_blobs(
    n_samples=24, centers=3,
    cluster_std=0.60, random_state=7,
)
```

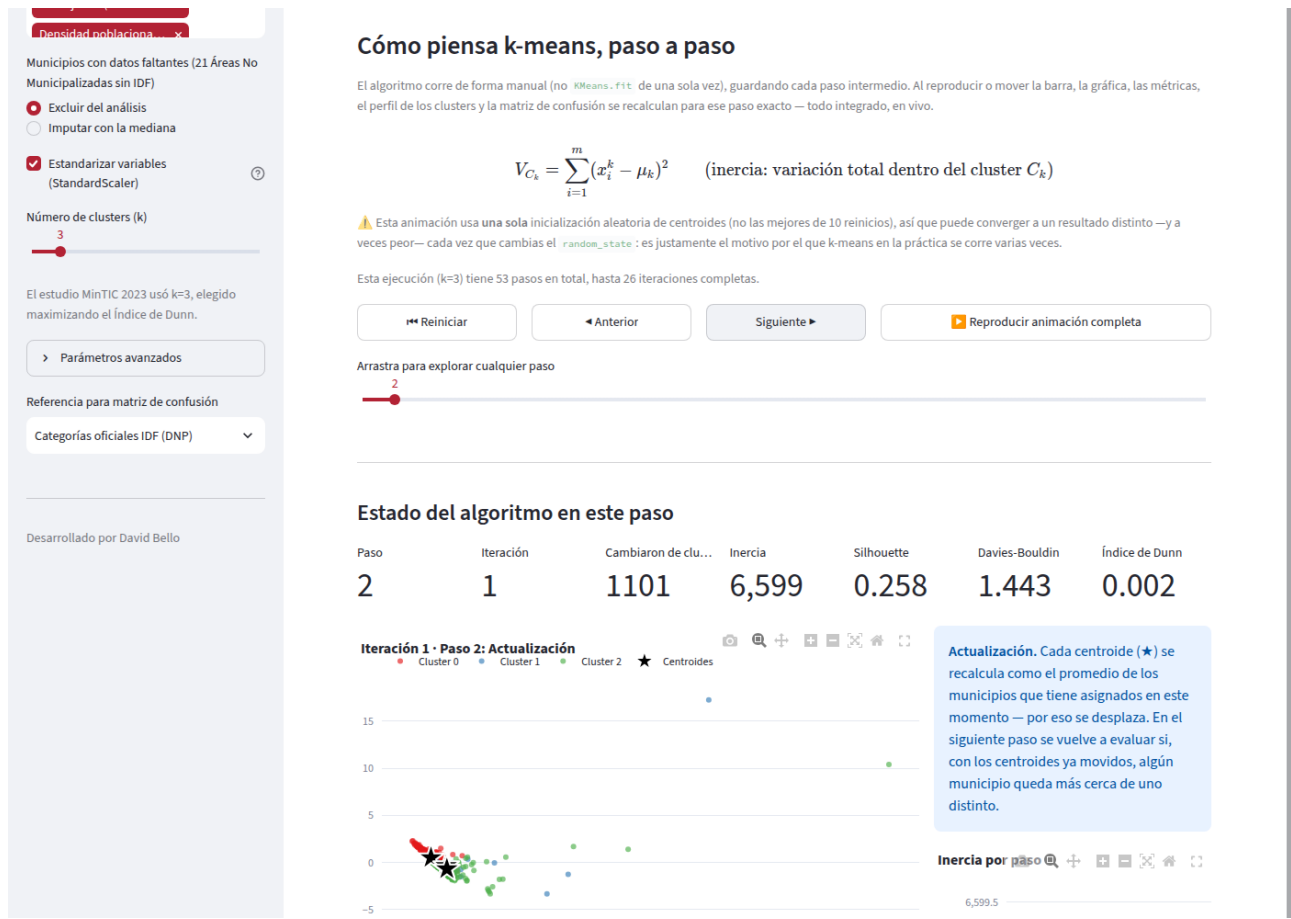
Paso 0 de 6

Paso 1 · Inicialización — se eligen $k=3$ puntos al azar como centroides:

Pestaña "Cómo funciona": código + gráfica del paso de inicialización.

3.2 Simulador interactivo — estado del algoritmo en vivo

Al recorrer la animación paso a paso sobre los 1.101 municipios, las métricas de calidad, la gráfica de clusters y la curva de inercia se recalculan para el paso exacto en el que uno se encuentre — este ejemplo muestra el paso 2 (actualización de centroides, iteración 1).



Matriz de confusión en este paso — vs. Categorías oficiales IDF (DNP)

Cada cluster se empareja con la categoría de referencia más frecuente dentro de él (algoritmo húngaro). Como el clustering aún puede estar cambiando, este emparejamiento también se recalcula en cada paso.

Accuracy (tras emparejar)

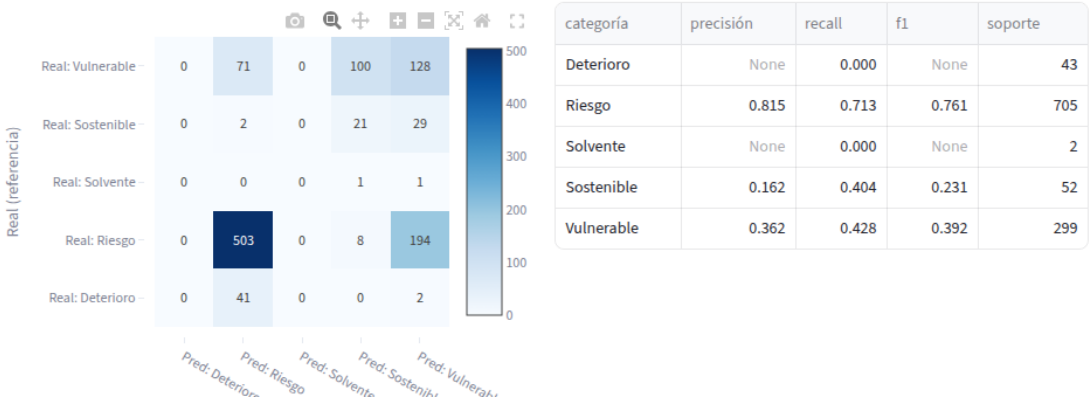
59.2%

ARI

0.232

NMI

0.203



Matriz de confusión vs. categorías oficiales del IDF, con métricas de acuerdo.

3.4 Repositorio en GitHub

Platform Solutions Resources Open Source Enterprise Pricing

Search / Sign in Sign up

daesbello / kmeans-municipios-colombia Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests Actions Projects Security and quality Insights

main 1 Branch 0 Tags

Go to file Code

daesbello

Agregar ARQUITECTURA.md con diseño técnico, dependencias y desple...

cc08910 · 26 minutes ago

7 Commits

.streamlit	Simulador K-Means de municipios de Colombia (IDF 2022)	2 hours ago
data	Simulador K-Means de municipios de Colombia (IDF 2022)	2 hours ago
src	Agregar módulo introductorio "Cómo funciona" antes del...	1 hour ago
.gitignore	Simulador K-Means de municipios de Colombia (IDF 2022)	2 hours ago
ARQUITECTURA.md	Agregar ARQUITECTURA.md con diseño técnico, depende...	26 minutes ago
INFORME.md	Agregar informe con metodología, resultados y matriz de...	1 hour ago
README.md	Agregar ARQUITECTURA.md con diseño técnico, depende...	26 minutes ago
app.py	Conectar el ejemplo sencillo con los controles de la barra ...	49 minutes ago
requirements.txt	Simulador K-Means de municipios de Colombia (IDF 2022)	2 hours ago

README

title	Simulador K-Means Municipios Colombia
emoji	📍
colorFrom	red

About

Simulador K-Means de municipios de Colombia (IDF 2022) — Streamlit

Readme Activity 0 stars 0 watching 0 forks Report repository

Releases

No releases published

Contributors 1

daesbello David Bello

Languages

Python 100%

Repositorio público daesbello/kmeans-municipios-colombia, con el historial de commits del proyecto.

4. Arquitectura de la aplicación

La aplicación es un **monolito Streamlit**: un solo proceso Python que sirve tanto la lógica como la interfaz web, sin backend/frontend separados ni base de datos — el dataset es un Excel versionado en el propio repositorio, y el estado de sesión vive en memoria (`st.session_state`). Se organiza en tres capas:

- **Datos** (`src/data_processing.py`): carga el Excel, renombra columnas, aplica la estrategia de datos faltantes (excluir o imputar con la mediana).
- **Motor de clustering** (`src/clustering.py`): k-medias implementado a mano, paso a paso (`kmeans_steps_from_matrix`), reutilizado tanto por el ejemplo sencillo en 2D como por `kmeans_animation` (que además estandariza y proyecta a 2D con PCA para las 5 variables reales); más métricas de calidad, curva del codo, y comparación contra referencias (matriz de confusión, ARI, NMI) vía el algoritmo húngaro.
- **UI / orquestación** (`app.py`): controles de la barra lateral y las tres pestañas de la app — llama a las capas anteriores y renderiza todo con Streamlit y Plotly.

4.1 Flujo de ejecución

Streamlit reejecuta el script completo en cada interacción del usuario. Al cambiar un control (p. ej. `k`), se recalcula `prepare_dataset` (cacheado) y luego `kmeans_animation/kmeans_steps_from_matrix` generan la lista completa de pasos para la nueva configuración (también cacheados). Como cambió la configuración, el paso actual se reinicia a 0, y se renderiza: gráfica, métricas, perfil de clusters y matriz de confusión — todo para ese paso exacto. Al presionar "Reproducir", un bucle de Python recorre los pasos restantes actualizando la interfaz con una pequeña pausa entre cada uno.

4.2 Dependencias

Paquete	Uso en esta app
streamlit	Framework de la interfaz web: sidebar, pestañas, widgets, cache, estado de sesión
pandas	Carga y manipulación del dataset (Excel → DataFrame, agrupaciones, filtros)
numpy	Álgebra vectorial: distancias, promedios, generación de números aleatorios
scikit-learn	StandardScaler, PCA, KMeans (curva del codo), métricas de clustering, <code>make_blobs</code>
scipy	<code>cdist</code> (distancias euclidianas) y el algoritmo húngaro para emparejar clusters
plotly	Todas las gráficas interactivas (dispersión, líneas, heatmap, barras)
openpyxl	Motor de pandas para leer el archivo .xlsx

No hay infraestructura adicional: sin Docker, sin base de datos, sin servicios externos en tiempo de ejecución.

4.3 Despliegue

El repositorio de GitHub está conectado directamente a **Streamlit Community Cloud** (gratuita); cada `git push` a `main` redespiega la app automáticamente (reinstala `requirements.txt` y reinicia el proceso). El repositorio también incluye la configuración YAML para Hugging Face Spaces por si se quiere usar en el

futuro, pero no se usó como plataforma final: al momento de construir esta app, Hugging Face Spaces exigía una suscripción paga para correr apps Python (solo el SDK "Static", sin backend, es gratuito), mientras que Streamlit Community Cloud sí permite apps Python gratis para repos públicos.

5. Metodología y resultados

El estudio de MinTIC (2023) agrupó los 1.101 municipios de Colombia en 3 clusters usando k-medias sobre variables socioeconómicas (población, ingresos, % de hogares con acceso a internet, desempeño institucional, densidad poblacional, IDF), para asignar plazos diferenciales de implementación del Marco de Referencia de Arquitectura Empresarial. Este trabajo reproduce esa metodología con las 5 variables disponibles en el dataset (no incluye acceso a internet), estandarizadas antes de calcular distancias.

De los 1.123 municipios del consolidado, 21 son Áreas No Municipalizadas sin Puntaje IDF y se excluyen por defecto (quedan 1.101, igual que el estudio original) — configurable para imputarlos con la mediana en su lugar.

5.1 Resultados con k=3 (configuración por defecto)

Métrica	Valor
Municipios usados	1.101 de 1.123
Inercia	3.538,96
Silhouette	0,294
Calinski-Harabasz	305,0
Davies-Bouldin	1,244
Índice de Dunn	0,001

El cluster 2 agrupa solo 7 municipios (las grandes ciudades, con población e ingresos varios órdenes de magnitud por encima del resto); los clusters 0 y 1 dividen al resto del país entre un grupo de condiciones más básicas y uno intermedio-avanzado — consistente con el hallazgo del estudio MinTIC de un cluster "avanzado" pequeño, uno intermedio y uno "básico".

5.2 Comparación contra referencias externas

Como k-means es no supervisado, se compara contra dos referencias: las **categorías oficiales del IDF** (DNP, una sola variable fiscal) y una **réplica del estudio MinTIC** (aproximación construida en esta app, no la clasificación original). El acuerdo contra el IDF es bajo (ARI 0,093) — esperable, ya que el IDF mide solo desempeño fiscal, mientras que el clustering es multivariado. El acuerdo contra la réplica MinTIC es mucho más alto (ARI 0,610), porque esa referencia se construye a partir de las mismas variables. El detalle completo, con matrices de confusión, precisión/recall/F1 y limitaciones, está en [INFORME.md](#) del repositorio.

6. Proceso de desarrollo asistido por IA

La aplicación se construyó de forma iterativa con **Claude Code** (Anthropic, modelo Claude Sonnet 5), a partir de instrucciones en lenguaje natural. En cada iteración se probó la app en un navegador (local y/o en producción) antes de darla por terminada. A continuación, el historial de prompts dados por el autor (transcritos tal como se escribieron) y qué se construyó en cada uno:

Prompt 1

“toca implementar el kmeans con el dataset de la salida y a partir del pdf de la entrada entonces la idea es hacer una aplicacion que esté en python y que tenga frontend para poder ver todo el poder del kmeans, que tenga documentacion esta aplicacion de como funciona y que sea estilo simulador y que permita ver matriz de confusion y metricas veme preguntando para que quede perfecta la app”

→ Primera versión de la app en Streamlit: carga de datos, k-means, curva del codo, matriz de confusión contra dos esquemas de referencia, documentación integrada.

Prompt 2

“hay algun mpc para dejar lista para publicarla en github y en esos servicios e punlvacion gratis, galta tambien ponerme mis datos”

→ Creación y push del repositorio en GitHub; datos de autoría en el README.

Prompt 3

“conevtemos”

→ Intento de conexión con Hugging Face (descartado por requerir plan pago) y despliegue final en Streamlit Community Cloud.

Prompt 4

“quiero hacerle una mejora tecnologica al app y es que se pueda ir moviendo y simulando con algun boton o escenarios y que exploque que esta pasando tecnicamente como si eso fuera un aula de clase interactiva”

→ Animación de k-means paso a paso con controles de reproducir/pausar/avanzar.

Prompt 5

“quisiera que unificaramos matriz de vonfusion metricas simulador y aula interactiva en uno solo de tal manera que se pueda ver todo integral a medida que se vagya ejecutando la simulacion”

→ Unificación de las tres vistas en una sola experiencia reactiva por paso.

Prompt 6

“metamosle un modulo antes de msimulador que explica graficamente como funciona kmanes sencillito pero que se entienda en codigo y graficamente que pasa”

→ Módulo "Cómo funciona": ejemplo mínimo en 2D con código y gráfica lado a lado.

Prompt 7

“la idea es que el como funciona se aritcule con la config de la izquierda tambien enronces si algo cambia cambia aca”

→ Conexión del ejemplo sencillo con k y random_state de la barra lateral.

Prompt 8

“y un pdf con todo organizado la docu links del repo de la app patnalalzos explicacion de lo que se hizo y obio mi nombre”

→ Este documento.

7. Conclusiones

- Es posible reproducir, con datos públicos, un agrupamiento de municipios de Colombia consistente en su estructura general con el reportado por MinTIC (2023): un grupo pequeño de grandes ciudades y una división del resto del país en dos niveles de desarrollo socioeconómico relativo.
- El agrupamiento multivariado (k-means) y la clasificación fiscal oficial (IDF-DNP) miden cosas distintas: el bajo acuerdo entre ambos (ARI 0,093) confirma que el desempeño fiscal por sí solo no determina el perfil socioeconómico general de un municipio.
- Separar el núcleo del algoritmo (`kmeans_steps_from_matrix`) del resto de la app permitió reutilizar exactamente el mismo código tanto en el ejemplo pedagógico de 2 dimensiones como en el caso real de 5 — reforzando que ambas vistas enseñan lo mismo.
- Todo el desarrollo se hizo de forma conversacional e iterativa con IA, validando cada cambio en el navegador (local o en producción) antes de continuar, lo que permitió detectar y corregir en el momento, por ejemplo, un problema de hot-reload en el despliegue.

David Bello — Septiembre de 2026

<https://github.com/daesbello/kmeans-municipios-colombia> ·
<https://kmeans-municipios-colombia.streamlit.app/>